

T.C.
MARMARA ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

WEB PROGRAMLAMA DERSİ

2PROJE RAPORU

2025-2026 Bahar Dönemi

Koku. — Parfüm Takip ve Sosyal Platform

<https://parfum.avesitesi.xyz>

PROJE EKİBİ

Burak Tuzcu

Emre Çoruhlu

Muhammet Batuhan Karanfil

Hazırlanma Tarihi: Haziran 2026

1. Projeye Genel Bakış

"Koku." projesi, parfüm meraklılarının koleksiyonlarını yönetebilecekleri, günlük parfüm kullanımlarını kayıt altına alabilecekleri, birbirleriyle etkileşime geçip topluluklar oluşturabilecekleri ve kişisel koku profillerine dayalı akıllı öneriler alabilecekleri kapsamlı bir web platformudur. Proje, 2025-2026 Bahar Dönemi Web Programlama dersi kapsamında geliştirilmiş olup gerçek bir sunucuya deploy edilerek canlıya alınmıştır.

Platform, 24.000'den fazla parfüm içeren gerçek bir veri seti üzerine inşa edilmiş olup ASP.NET Core 9 MVC mimarisiyle geliştirilmiştir. Sosyal medya benzeri etkileşim katmanı, akıllı içerik tabanlı öneri motoru ve tam işlevsel CI/CD pipeline ile proje akademik düzeyin ötesinde üretim kalitesine taşınmıştır.

1.1 Proje Künyesi

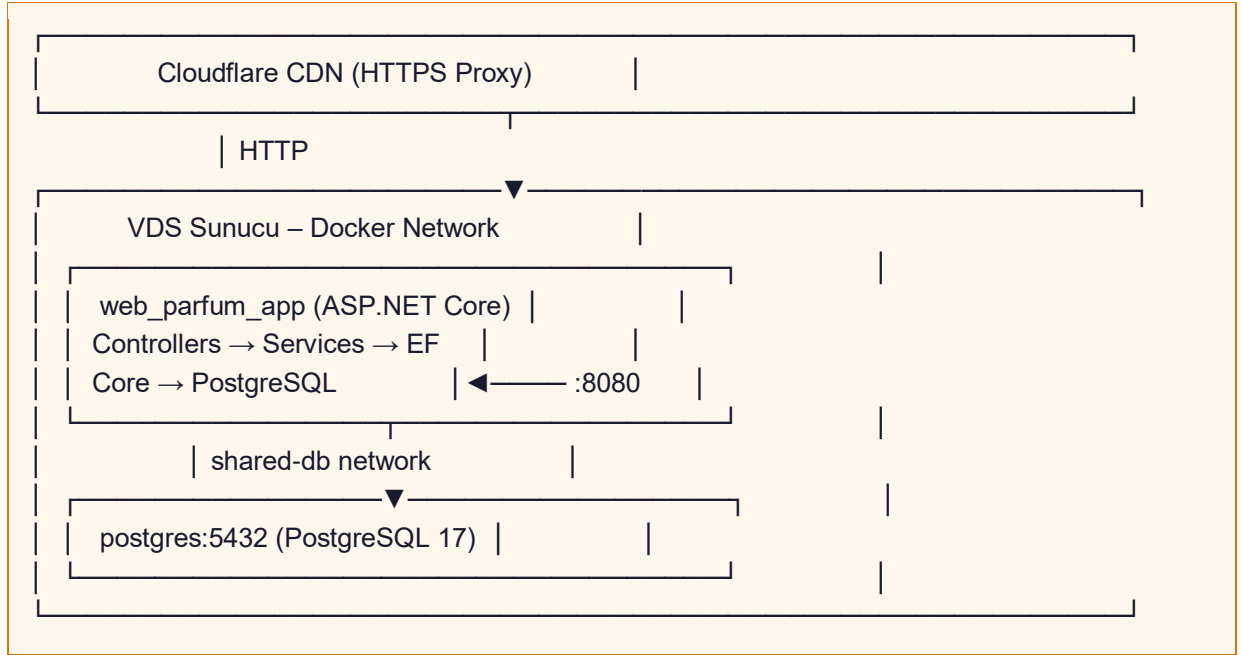
Alan	Bilgi
Proje Adı	Koku. — Parfüm Takip ve Sosyal Platform
Canlı URL	https://parfum.avesitesi.xyz
Ders	Web Programlama (2025-2026 Bahar)
Ekip Üyeleri	Burak Tuzcu, Emre Çoruhlu, Muhammet Batuhan Karanfil
Teknoloji Stack	ASP.NET Core 9 MVC, PostgreSQL 17, Docker, GitHub Actions
Veri Boyutu	~24.000 parfüm (fra_cleaned.csv)
Deployment	VDS + Cloudflare Proxy (HTTPS)

2. Sistem Mimarisi

2.1 Genel Mimari Yaklaşım

Proje, katmanlı (layered) bir mimari anlayışıyla geliştirilmiştir. ASP.NET Core MVC'nin sunduğu Model-View-Controller deseni temel alınmış; üzerine servis katmanı, filtre katmanı ve yardımcı sınıflar (helpers) eklenmiştir. Tüm uygulama Docker container içinde çalışmakta ve Cloudflare CDN/proxy katmanının arkasında konumlanmaktadır.

2.2 Katman Diyagramı



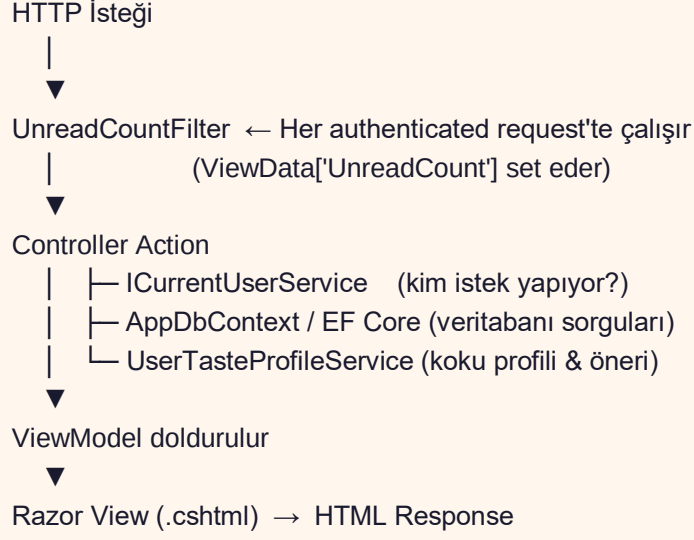
2.3 MVC Katman Yapısı

Projenin klasör ve katman yapısı aşağıdaki gibi organize edilmiştir:

Katman / Klasör	İçerik	Sorumluluk
Controllers/	12 controller sınıfı	HTTP istek yönetimi, iş mantığı koordinasyonu
Models/	13 entity sınıfı	Veritabanı modelleri
ViewModels/	15+ ViewModel	View'a veri taşıma
Views/	Tüm .cshtml dosyaları	UI katmanı (Razor)
Services/	3 servis sınıfı	İş mantığı soyutlaması
Data/	AppDbContext, DbInitializer	Veri erişim katmanı
Helpers/	AccordVisual, NoteVisual, SearchNormalizer vb.	Yardımcı fonksiyonlar
Filters/	UnreadCountFilter	Action filter (bildirim sayısı)

Migrations/	3 migration dosyası	DB şema versiyonlama
wwwroot/	CSS, JS dosyaları	Statik varlıklar

2.4 Veri Akış Diyagramı



3. Veritabanı Tasarımı

3.1 EF Core Code-First Yaklaşımı

Proje, Entity Framework Core 9.0 ile Code-First yaklaşımını benimsemiştir. Tüm tablo şeması, C# model sınıfları aracılığıyla tanımlanmış ve Migrations sistemi üzerinden PostgreSQL 17 veritabanına uygulanmıştır. ApplicationDbContext sınıfı, DbSet özelliklerini ve ilişki konfigürasyonlarını barındırmaktadır.

3.2 Varlık (Entity) Listesi

Entity	Temel Alanlar	İlişkiler
User	Id, Username, Email, PasswordHash, Bio, Avatar, CreatedAt	Follows, Collections, DailyLogs, Reviews, Likes, Messages, Notifications
Perfume	Id, Name, Brand, Country, Gender, Year, TopNotes, MiddleNotes, BaseNotes, Accord1-5, RatingValue, RatingCount, ImageUrl, SearchKey (GENERATED)	Collections, DailyLogs, Reviews, Likes, CommunityPosts
Collection	Id, UserId, PerfumeId, Status (owned/wishlist/tried), BottleLevel, CreatedAt, UpdatedAt	User, Perfume
DailyLog	Id, UserId, PerfumeId, Date, Sprays, Note, CreatedAt	User, Perfume
Review	Id, UserId, PerfumeId, Rating (1-5), Body, CreatedAt	User, Perfume
Like	Id, UserId, PerfumeId, CreatedAt	User, Perfume
Follow	Id, FollowerId, FollowingId, CreatedAt	User (x2)
Community	Id, Name, Description, Image, OwnerId, CreatedAt	User (Owner), Members, Posts
CommunityMember	Id, CommunityId, UserId, Role (admin/member), JoinedAt	Community, User
CommunityPost	Id, CommunityId, UserId, PerfumeId?, Body, CreatedAt	Community, User, Perfume?
Message	Id, SenderId, RecipientId, Body, IsRead, CreatedAt	User (x2)
Notification	Id, RecipientId, ActorId, Type, PerfumeId?, CommunityId?, IsRead, CreatedAt	User (x2)

3.3 Unique Constraint'ler

ApplicationDbContext.OnModelCreating() içinde aşağıdaki benzersizlik kısıtlamaları tanımlanmıştır:

- User.Username → INDEX UNIQUE
- User.Email → INDEX UNIQUE

- Review(UserId, PerfumeId) → INDEX UNIQUE (kullanıcı başına bir yorum)
- Like(UserId, PerfumeId) → INDEX UNIQUE
- Collection(UserId, PerfumeId, Status) → INDEX UNIQUE (aynı status iki kez olamaz)
- Follow(FollowerId, FollowingId) → INDEX UNIQUE
- CommunityMember(CommunityId, UserId) → INDEX UNIQUE

3.4 PostgreSQL Trigram Arama İndeksi

Üçüncü migration (AddPerfumeSearchIndex) ile PostgreSQL'e özgü gelişmiş arama altyapısı kurulmuştur:

- CREATE EXTENSION unaccent ve pg_trgm eklentileri etkinleştirilmiştir.
- **perfume_normalize()** adlı IMMUTABLE SQL fonksiyonu tanımlanmıştır: aksan kaldırma (unaccent), lowercase dönüşümü ve alfanümerik olmayan karakterlerin temizlenmesi.
- **SearchKey** kolonu: Name + Brand + TopNotes + MiddleNotes + BaseNotes + Accord1..5 birleşiminin normalize edilmiş hâli. GENERATED ALWAYS AS STORED olarak tanımlıdır; yazma operasyonlarında otomatik güncellenir.
- **SearchKeyName** kolonu: Sadece Name + Brand üzerinden. Sıralama ve prefix önceliği için ayrı tutulmuştur.
- GIN trigram indeksleri (ix_perfumes_search_key_trgm, ix_perfumes_search_key_name_trgm) ile tiplara karşı dayanıklı, kelime sırasından bağımsız, Türkçe aksanlara duyarsız arama sağlanmıştır.

3.5 Migration Geçmişi

Migration Adı	Yapılan Değişiklik
20260508195003_InitialCreate	Tüm tabloların ilk oluşturulması, FK'lar, temel indeksler
20260525131138_CollectionMultiStatus	Collection tablosundaki (UserId, PerfumeId) unique kısıtlaması (UserId, PerfumeId, Status) olarak genişletildi; aynı parfüm birden fazla statüde olabilsin diye
20260525175130_AddPerfumeSearchIndex	unaccent & pg_trgm extension, perfume_normalize() fonksiyonu, SearchKey ve SearchKeyName GENERATED STORED kolonları, GIN trigram indeksleri

3.6 Seed Verisi

DbInitializer.Initialize() metodu, uygulama ilk kez ayağa kalktığında çalışarak veritabanını hazır hâle getirir:

- fra_cleaned.csv dosyasından (~24.000 satır, ISO-8859-1 encoding) parfüm verisi okunur ve 500'lük batch'ler hâlinde Perfumes tablosuna aktarılır.
- Tablo doluysa mock veri kontrolü yapılır: 6 demo kullanıcı (emre, ayse, mehmet, zeynep, can, lale), koleksiyonlar, yorumlar, günlük loglar, beğeniler, takip ilişkileri, bildirimler, 2 topluluk ve mesajlar oluşturulur.
- Tüm demo kullanıcıların şifresi 'password123' olarak BCrypt ile hash'lenmiştir.

4. Kimlik Doğrulama ve Yetkilendirme

4.1 Cookie Tabanlı Authentication

Proje, ASP.NET Core'un Cookie Authentication mekanizmasını kullanmaktadır. Program.cs içinde şu şekilde yapılandırılmıştır:

```
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options => {
        options.LoginPath = "/Auth/Login";
        options.LogoutPath = "/Auth/Logout";
        options.ExpireTimeSpan = TimeSpan.FromDays(7);
        options.SlidingExpiration = true;
    });
```

Kullanıcı başarıyla giriş yaptığında ClaimsIdentity nesnesi oluşturulur ve HTTP cookie içine şifreli olarak yerleştirilir. Cookie, 7 gün geçerliliğini korumakta; sliding expiration sayesinde aktif kullanımda otomatik uzamaktadır.

4.2 AuthController — Kayıt ve Giriş Akışı

AuthController sınıfı kayıt (Register) ve giriş (Login) işlemlerini yönetir:

- **Kayıt:** E-posta ve kullanıcı adının veritabanında benzersiz olup olmadığı kontrol edilir. Şifre, BCrypt.Net kütüphanesi ile hash'lenerek kaydedilir. Başarılı kayıt sonrasında kullanıcı otomatik olarak oturum açılır.
- **Giriş:** Kullanıcı e-posta adresi ile sorgulanır; BCrypt.Verify() ile şifre doğrulanır. Başarılı doğrulama sonrası ClaimTypes.NameIdentifier, ClaimTypes.Name ve ClaimTypes.Email claim'leri içeren bir kimlik oluşturularak HttpContext.SignInAsync çağrılır.
- **Çıkış:** HttpContext.SignOutAsync() ile cookie temizlenir, kullanıcı Login sayfasına yönlendirilir.

4.3 ICurrentUserService

Aktif kullanıcının kimliğini controller'lardan bağımsız biçimde elde etmek için ICurrentUserService arayüzü ve CurrentUserService implementasyonu oluşturulmuştur. Bu servis, IHttpContextAccessor üzerinden ClaimTypes.NameIdentifier claim'ini okuyarak UserId, Username ve IsAuthenticated özelliklerini sağlar. Tüm [Authorize] controller'lara constructor injection ile enjekte edilir.

4.4 Yetkilendirme Katmanları

Projede iki düzeyli yetkilendirme modeli uygulanmaktadır:

4.4.1 Controller Seviyesi [Authorize]

CollectionController, CommunitiesController, DailyLogController, FeedController, FollowsController, LikesController, MessagesController, NotificationsController ve ReviewsController sınıfları [Authorize] attribute ile işaretlenmiştir. Kimlik doğrulaması

yapılmamış bir kullanıcı bu sayfalara erişmeye çalıştığında /Auth/Login adresine yönlendirilir. PerfumesController public bırakılmıştır, böylece anonim kullanıcılar parfüm kataloğunu görebilir.

4.4.2 Topluluk Rol Tabanlı Yetkilendirme (Admin / Member)

Topluluklar özelinde iki farklı kullanıcı rolü tanımlanmıştır:

Rol	Atanma Zamanı	Yetkiler
admin	Topluluğu oluşturan kullanıcıya otomatik atanır	Post silme (tüm üyelerin postları dahil), topluluktan ayıramaz, topluluğu temsil eder
member	JoinToggle ile üye olduğunda atanır	Post oluşturma, üye listesini görme, gönderileri beğenme/yorum yapma

CommunitiesController içindeki rol denetimi aşağıdaki senaryolarda devreye girer:

- DeletePost: Post sahibi OR community owner silebilir (post.UserId != userId && post.Community.OwnerId != userId kontrolü).
- JoinToggle: Owner topluluğunu terk edemez; üyeler serbestçe ayrılabilir.
- CreatePost: Yalnızca üyeler (CommunityMembers tablosunda kaydı olanlar) gönderi paylaşabilir; üye olmayanlar Forbid() döner.
- Details sayfasında IsOwner bayrağına göre silme butonları koşullu render edilir.

4.4.3 Sahiplik Tabanlı Yetkilendirme

Rol dışında, kaynak sahipliği de kontrol edilmektedir:

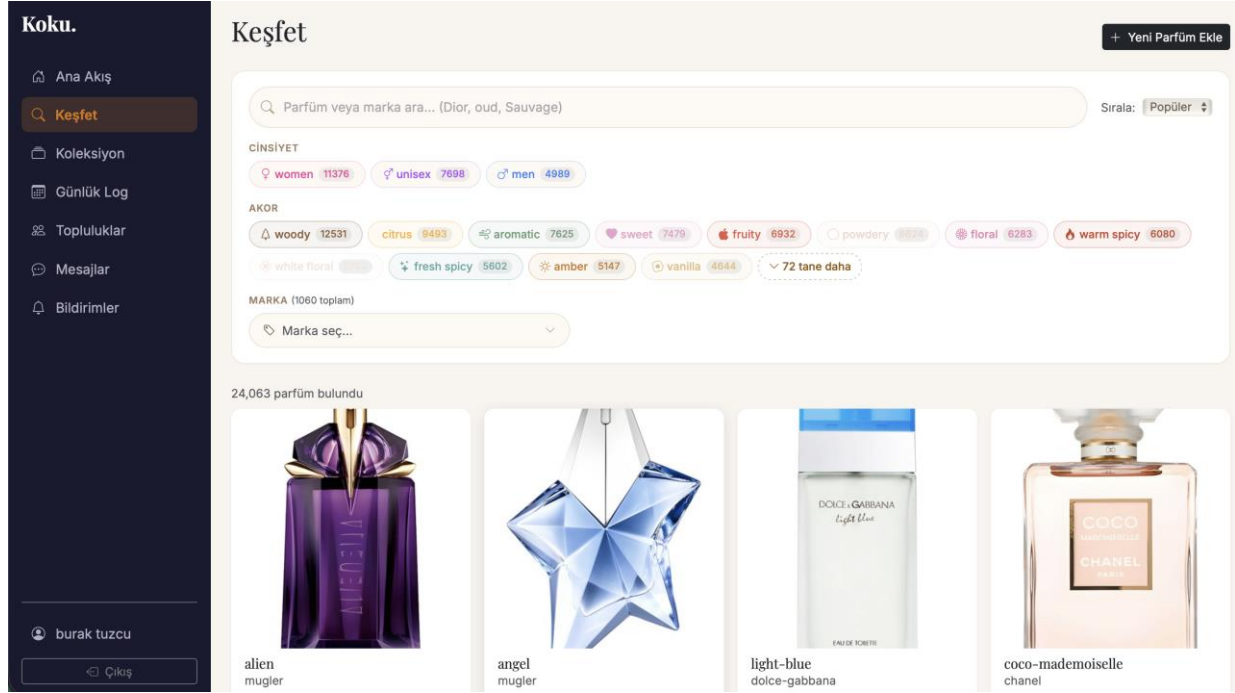
- DailyLog silinirken: log.UserId != userId ise Forbid() döner.
- Review silinirken: review.UserId != userId ise Forbid() döner.
- Mesajlar: Yalnızca gönderen veya alıcı konuşmayı görebilir.
- Bildirimler: notif.RecipientId != userId ise Forbid() döner.

4.5 Cloudflare Güvenlik Katmanı

Production ortamında Cloudflare proxy aktiftir. SSL/TLS Flexible modu ile ziyaretçi-Cloudflare arası HTTPS, Cloudflare-sunucu arası HTTP olarak çalışmaktadır. Program.cs içinde UseForwardedHeaders() middleware'i, Cloudflare'in X-Forwarded-Proto ve X-Forwarded-For başlıklarını okuyarak gerçek IP ve protokolü ASP.NET Core'a iletmektedir. Bu sayede cookie'deki Secure flag doğru kurulmakta ve redirect loop oluşmamaktadır.

5. Uygulama Özellikleri ve İş Mantığı

5.1 Parfüm Kataloğu ve Arama

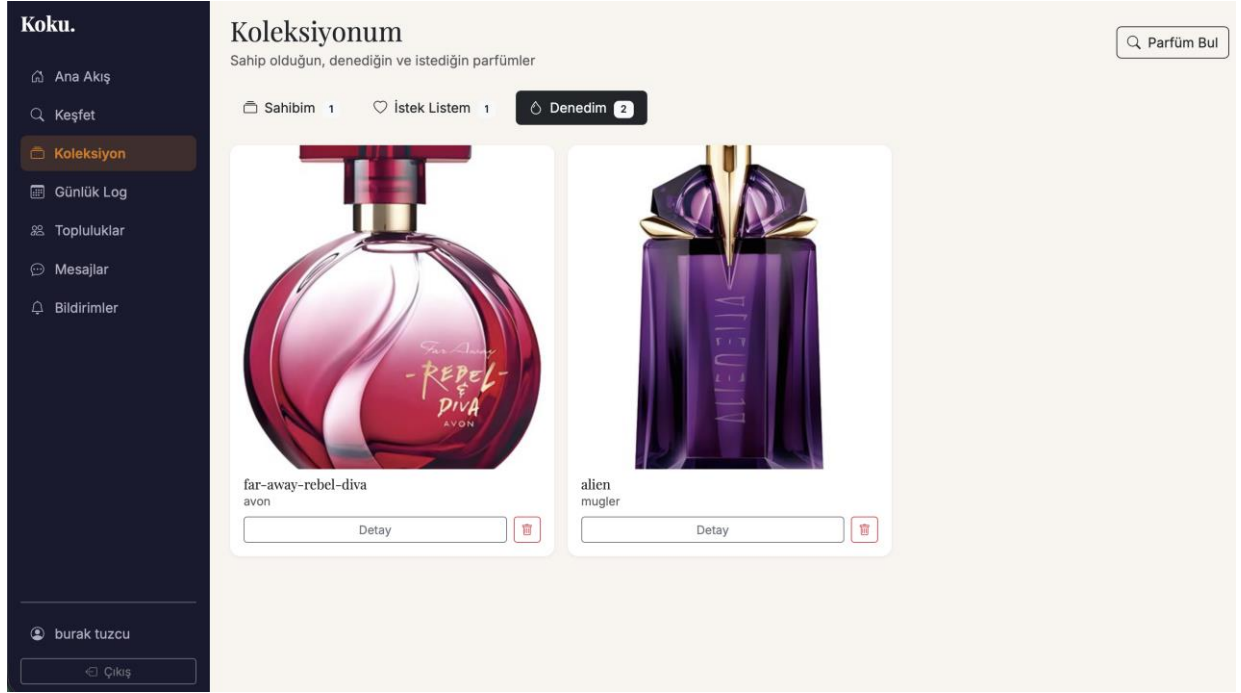


PerfumesController ve PerfumeSearchQuery helper sınıfı, gelişmiş arama işlevselliği sunar. Arama sorgusu SearchNormalizer.Tokenize() ile token'lara ayrılır; her token için hem ILIKE substring araması hem de TrigramsWordSimilarity > 0.4 eşliğiyle trigram benzerlik araması yapılır. Token'lar AND mantığıyla birleştirilir; böylece "dior sauvage" sorgusu hem isim hem marka eşleşmesi sağlar.

Keşfet sayfasında AJAX tabanlı anlık arama uygulanmıştır: kullanıcı yazarken 300ms debounce ile /Perfumes/Search endpoint'i çağrılır, X-Requested-With XMLHttpRequest başlığı tespit edilince yalnızca _PerfumeGrid partial view'ı döndürülür. Bu sayede sayfa tam yenilenmeden, input focus'u kaybolmadan grid güncellenir.

- Filtreler: Marka, Cinsiyet, Accord, Yıl
- Sıralama: Popüler (ratingCount), Puan (ratingValue), Yeni (year)
- Sayfalama: 24 kayıt/sayfa
- Autocomplete: DailyLog, CommunityPost ve Feed modal'larında parfüm arama widget'ı

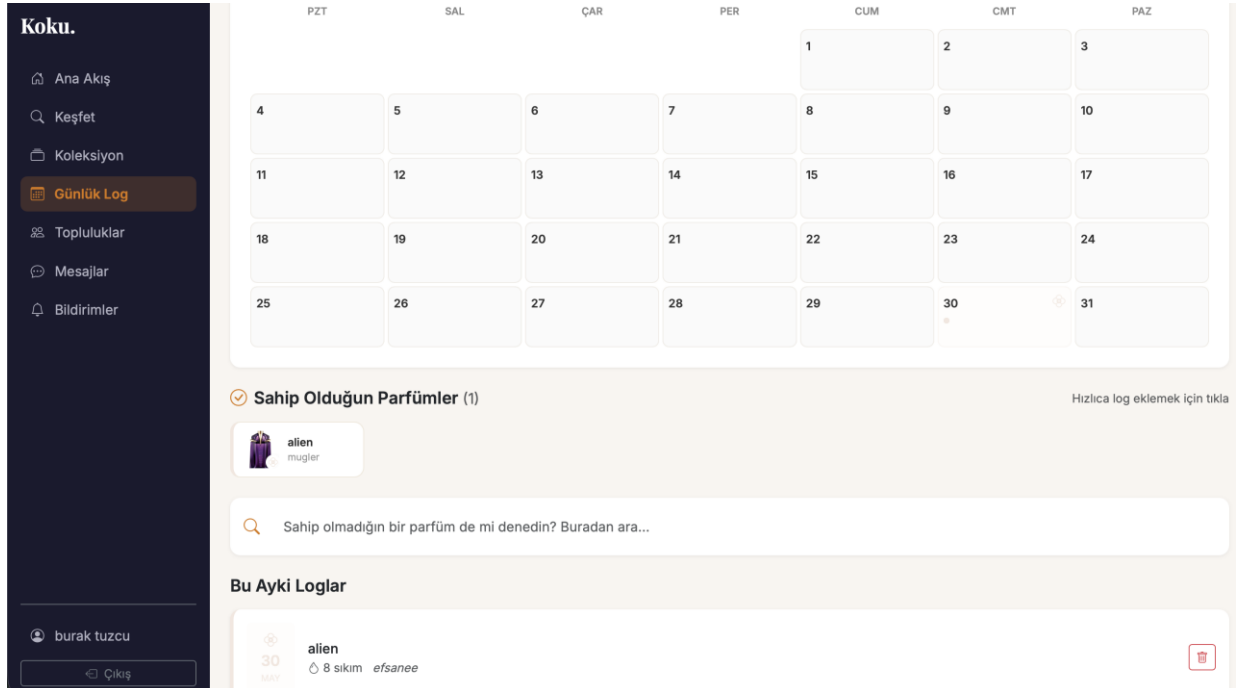
5.2 Koleksiyon Yönetimi



Her kullanıcı, parfümleri üç farklı statüde koleksiyonuna ekleyebilir: owned (sahip), wishlist (istek listesi) ve tried (denedim). Aynı parfüm birden fazla statüde bulunabilir (CollectionMultiStatus migration ile sağlanmıştır). Owned statüsünde şişe doluluk yüzdesi (BottleLevel) de takip edilir; kullanıcı slider ile 0-100 arası değer girebilir.

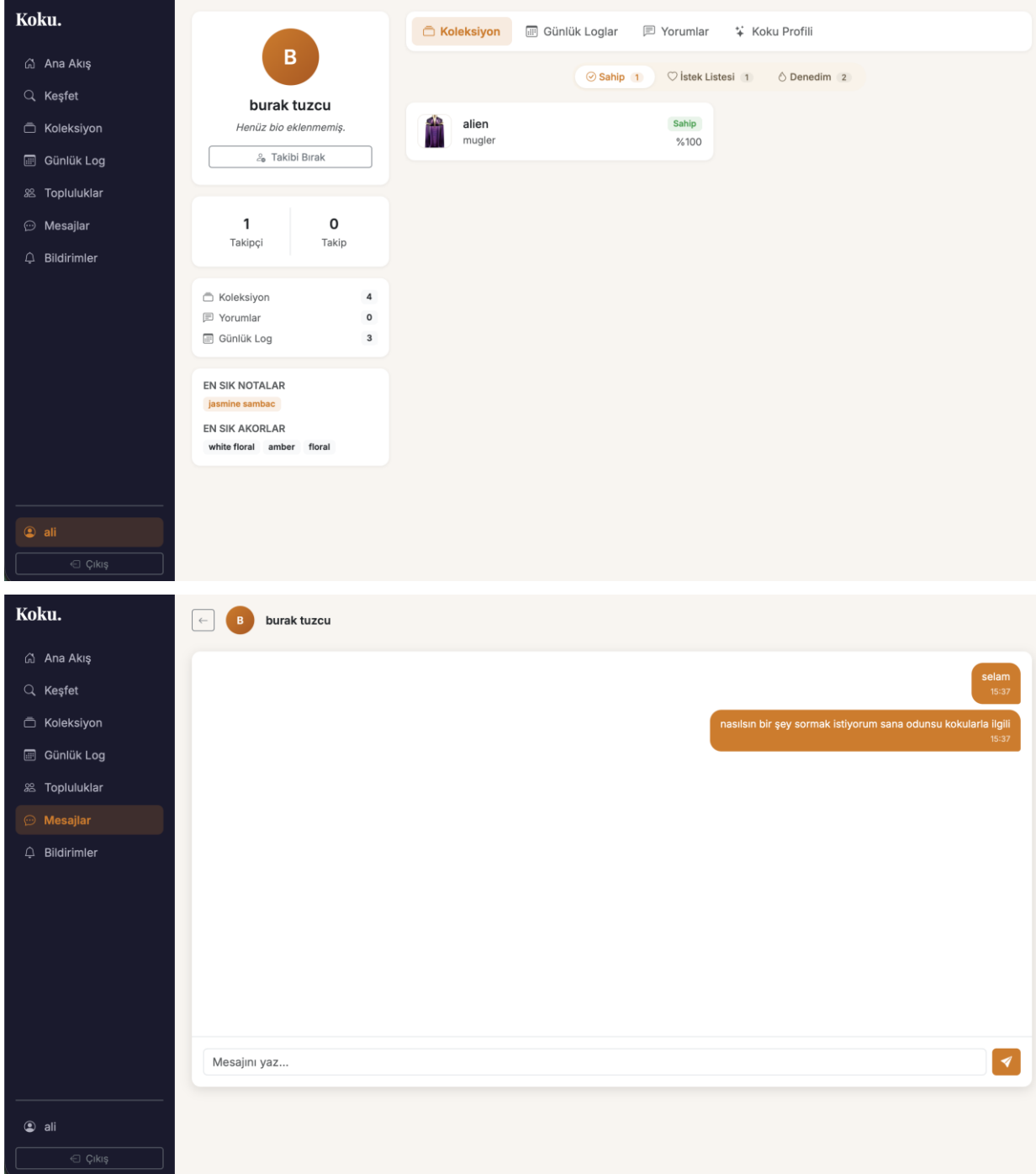
Toggle endpoint'i (Collection/Toggle), aynı URL üzerinden hem ekleme hem çıkarma işlemi yapar: kayıt mevcutsa siler, yoksa oluşturur. Bu sayede frontend'de karmaşık durum yönetimine gerek kalmaz.

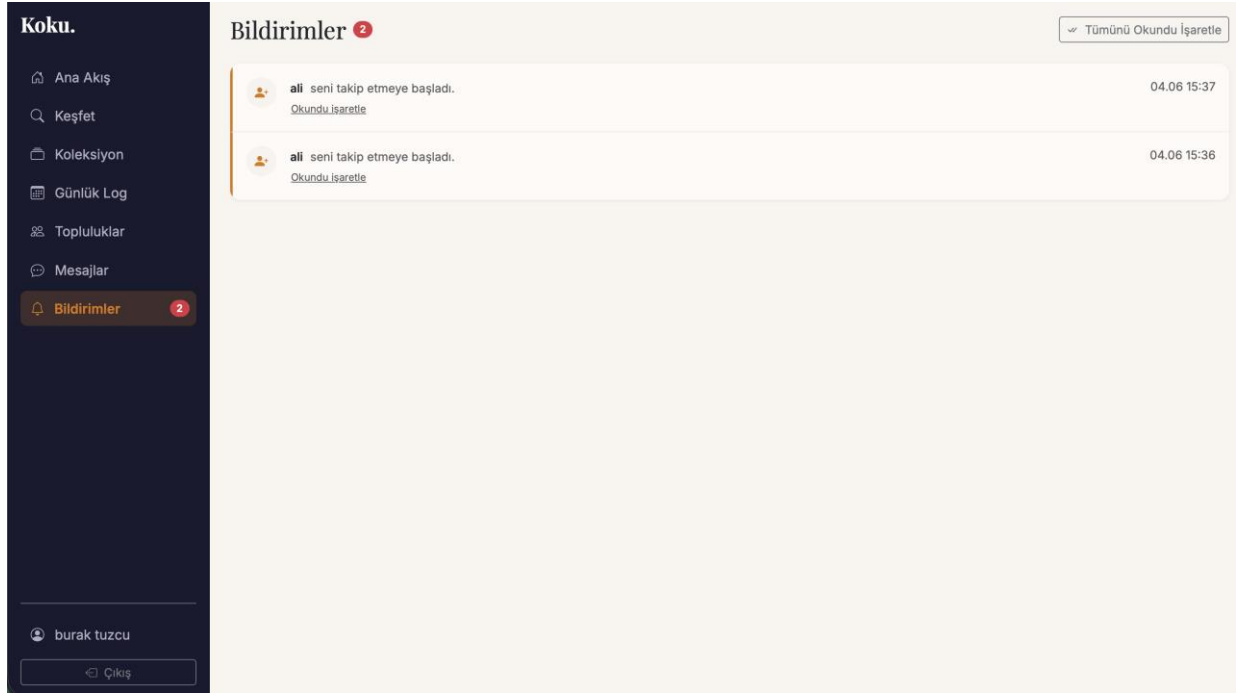
5.3 Günlük Log ve Takvim



DailyLogController, kullanıcıların hangi gün hangi parfümü kaç sprey kullandıklarını kayıt altına almasını sağlar. Takvim görünümü, aylık bazda ISO hafta başlangıcı (Pazartesi) ile grid olarak render edilir. Her takvim hücresi, o güne ait parfümün birincil accord rengini arka plan olarak kullanır; koku profiline göre renklenen bir parfüm takvimi oluşur.

5.4 Sosyal Katman

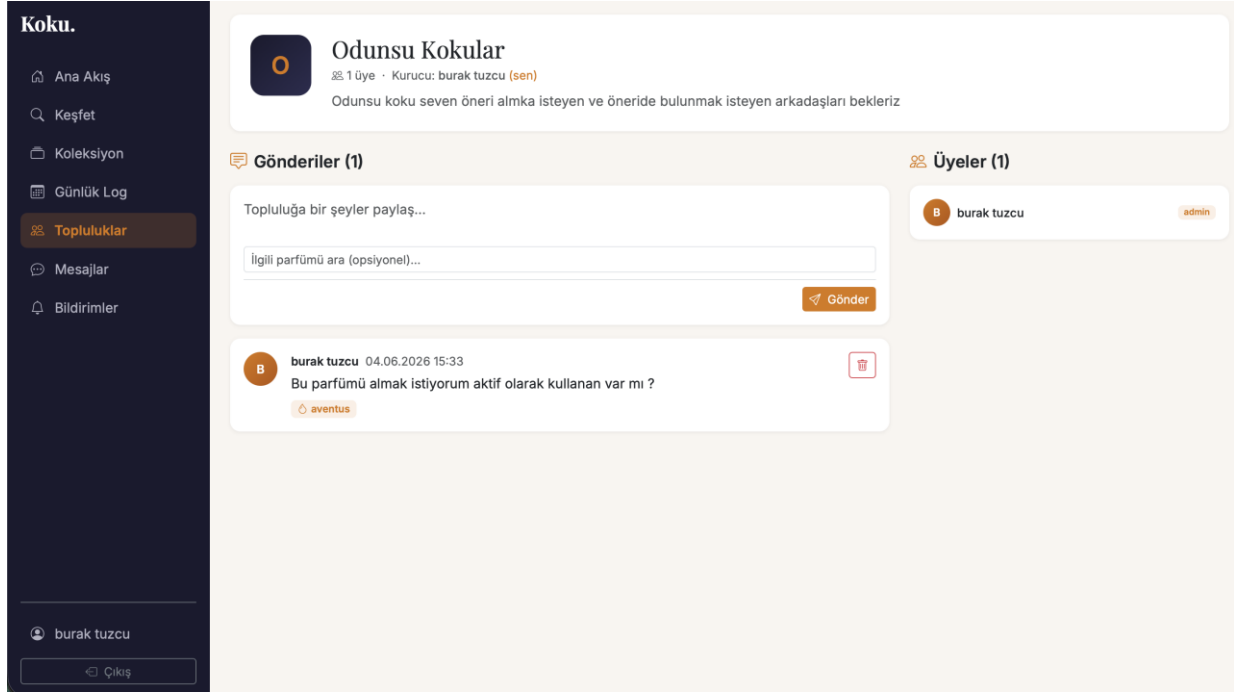




Proje, tam işlevsel bir sosyal ağ altyapısı içermektedir:

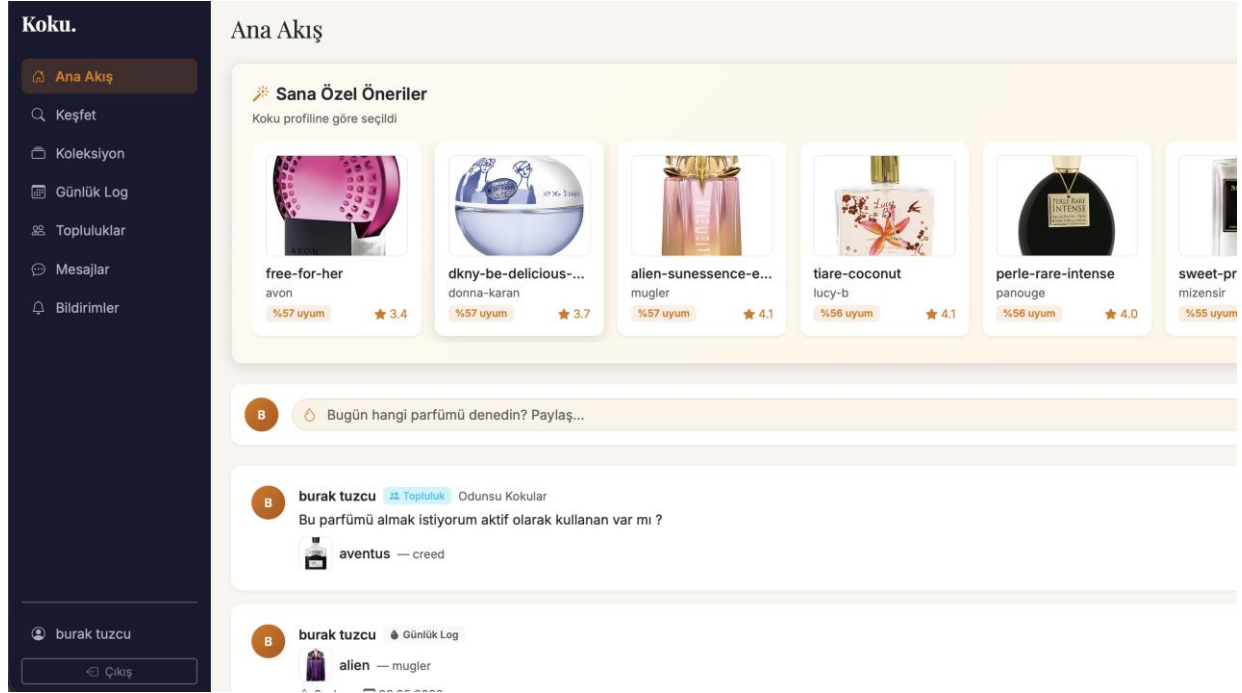
- **Takip (Follow/Unfollow):** Kullanıcılar birbirini takip edebilir. Feed, takip edilen kullanıcıların aktivitelerini gösterir. Takip edildiğinde bildirim oluşturulur.
- **Beğeni (Like Toggle):** AJAX tabanlı anında güncelleme. Fetch API ile POST; JSON yanıtı alınca buton görünümü ve sayaç JavaScript ile güncellenir, sayfa yenilenmez.
- **Yorum (Review):** Kullanıcı başına bir yorum limiti. Yorum sonrasında o parfüme sahip diğer kullanıcılara (owned statüs, son 10 kişi) bildirim gönderilir. Rating değişimi üzerine Perfume tablosundaki RatingValue ve RatingCount transaction içinde güncellenir.
- **Mesajlaşma:** Kullanıcılar arası özel mesajlaşma. Okunmamış mesajlar işaretlenir; konuşmaya girildiğinde otomatik okundu olarak işaretlenir.
- **Bildirimler:** follow, like, review tiplerinde bildirim üretilir. UnreadCountFilter action filter'ı her authenticated request'te sidebar'daki badge sayısını ViewData ile taşır.

5.5 Topluluklar



Kullanıcılar topluluk oluşturabilir, katılabilir ve gönderi paylaşabilir. Topluluk kurucusu otomatik admin rolü alır. Admin ve member rollerine bağlı yetkilendirme daha önce Bölüm 4'te anlatılmıştır. Topluluk gönderileri opsiyonel olarak belirli bir parfüme etiketlenebilir; bu parfüm adı doğrudan parfüm detay sayfasına bağlantı verir.

5.6 Akıllı Koku Profili ve Öneri Motoru



UserTasteProfileService, kullanıcının tüm etkileşimlerinden ağırlıklı bir koku profili çıkarır ve hibrit içerik/işbirlikçi öneri motoru çalıştırır. Sinyal ağırlıkları:

Sinyal Türü	Ağırlık
Owned (koleksiyon)	+2.5
Like (beğeni)	+2.0
Wishlist	+1.5
Review 5★	+3.0
Review 4★	+1.5
Review 3★	0.0 (nötr)
Review 2★	-1.5
Review 1★	-3.0 (negatif profil)
Tried (denedim)	+0.5
DailyLog (günlük)	+0.3 × kayıt sayısı (max 3.0)

Pozitif/negatif accord vektörleri oluşturulduktan sonra, kullanıcının daha önce etkileşmediği parfümlerle cosine similarity hesaplanır. Negatif accord cezası uygulanır. İşbirlikçi filtreleme için en az 2 ortak parfümü beğenen benzer kullanıcılar belirlenir; onların önerdiği parfümler collabScore alır. Nihai skor: %70 content + %30 collaborative.

6. UI ve Layout Yapısı

6.1 _Layout.cshtml — Ortak Şablon

Tüm sayfalar _Layout.cshtml üzerinden tek bir şablon kullanır. Layout dosyası, kimlik doğrulama durumuna göre iki farklı görünüm sunar: giriş yapmamış kullanıcılar için sade tam sayfa görünüm; giriş yapmış kullanıcılar için sabit sol sidebar, mobil üst bar ve ana içerik alanından oluşan panel yapısı.

Sidebar navigasyonunda sekiz ana bağlantı bulunmaktadır: Ana Akış, Keşfet, Koleksiyon, Günlük Log, Topluluklar, Mesajlar, Bildirimler ve Profil. Bildirim bağlantısının yanında, UnreadCountFilter tarafından ViewData["UnreadCount"] aracılığıyla sağlanan okunmamış bildirim sayısı badge olarak gösterilmektedir.

6.2 ViewBag / ViewData Kullanımı

ViewData Anahtarı	Kullanım Amacı
ViewData["Title"]	Her sayfada sekme başlığı
ViewData["ActivePage"]	Sidebar'da aktif menü öğesini vurgulamak için
ViewData["UnreadCount"]	UnreadCountFilter tarafından set edilen okunmamış bildirim sayısı

6.3 ViewModel Kullanımı

Tüm view'lar kesinlikle ViewModel pattern'ini kullanmaktadır. Model sınıfları doğrudan view'a aktarılmaz; her sayfanın ihtiyacına özel ViewModel sınıfları oluşturulmuştur:

- PerfumeIndexViewModel — Arama, filtreleme, sıralama, sayfalama parametreleri ve sonuç listesi
- PerfumeDetailViewModel — Parfüm, yorumlar, beğeni sayısı, kullanıcının koleksiyon statüsü
- FeedViewModel — Feed öğeleri, popüler fallback, kişiselleştirilmiş öneriler
- ProfileViewModel — Kullanıcı bilgileri, koleksiyon/log/yorum sekmesi verileri, koku profili
- DailyLogIndexViewModel — Takvim yapısı, gün bazında gruplu loglar
- CommunityDetailViewModel — Topluluk, üyeler, gönderiler, IsOwner bayrağı
- CollectionIndexViewModel — Tab (owned/wishlist/tried), öğe listesi, sayımlar
- MessagesIndexViewModel, ConversationViewModel — Konuşma listesi ve tekil konuşma

6.4 Partial View'lar

_PerfumeGrid.cshtml, Keşfet sayfasında hem tam sayfa hem de AJAX yanıtı olarak kullanılmaktadır. Request.Headers.XRequestedWith == "XMLHttpRequest" kontrolü ile aynı action method'u koşullu olarak partial view döndürmektedir.

6.5 Tasarım Sistemi

site.css dosyasında özel bir tasarım sistemi kurulmuştur. CSS custom property'leri (--parfum-amber, --parfum-dark, --parfum-bg), Bootstrap 5.3'ün üzerine bindirilmiş bileşenler ve tam responsive layout içermektedir. Mobil için hamburger menü ve backdrop overlay özel JavaScript ile implement edilmiştir. 991px altında sidebar gizlenir, mobil topbar görünür hale gelir.

AccordionVisual ve NoteVisual yardımcı sınıfları, parfüm akor/nota isimlerini renk ve Bootstrap Icons sınıflarına eşlemektedir. Bu sayede takvim hücreleri, filtre chip'leri ve nota badge'leri her accordun karakterine uygun renk alır (örn. woody → kahverengi, floral → pembe, citrus → sarı).

7. Validasyonlar

7.1 Data Annotations

Tüm form girişleri Data Annotations attribute'ları ile sunucu tarafında doğrulanmaktadır. ModelState.IsValid kontrolü controller'larda yapılmakta; hata durumunda kullanıcı aynı form sayfasına hata mesajlarıyla birlikte döndürülmektedir.

ViewModel / Model	Alan	Uygulanan Kural
LoginViewModel	Email	[Required], [EmailAddress]
LoginViewModel	Password	[Required], [DataType(Password)]
RegisterViewModel	Username	[Required], [MaxLength(50)]
RegisterViewModel	Email	[Required], [EmailAddress]
RegisterViewModel	Password	[Required], [MinLength(6)]
PerfumeCreateViewModel	Name	[Required], [MaxLength(200)]
PerfumeCreateViewModel	Brand	[Required], [MaxLength(100)]
PerfumeCreateViewModel	Year	[Range(1800, 2100)]
PerfumeCreateViewModel	ImageUrl	[Url]
CommunityCreateViewModel	Name	[Required], [MaxLength(100)]
CommunityCreateViewModel	Description	[MaxLength(500)]
CommunityCreateViewModel	ImageUrl	[Url]
Collection (model)	Status	[Required], [MaxLength(20)]
Review (model)	Rating	[Range(1, 5)]
User (model)	Username	[Required], [MaxLength(50)]
User (model)	Email	[Required], [MaxLength(255)]

7.2 Controller Seviyesinde Ek Validasyonlar

Data Annotations dışında, iş mantığı gereksinimleri doğrultusunda controller'larda ek doğrulamalar yapılmaktadır:

- AuthController: Kayıt sırasında e-posta ve kullanıcı adı benzersizliği veritabanı sorgusuyla kontrol edilir (AnyAsync).
- ReviewsController: Kullanıcı başına tek yorum kontrolü yapılır.

- **CollectionController:** Status değerinin geçerli olup olmadığı ValidStatuses HashSet ile kontrol edilir; BottleLevel Math.Clamp(0,100) ile sınırlandırılır.
- **CommunitiesController:** Topluluk adı benzersizliği ILike ile case-insensitive kontrol edilir.
- **FollowsController:** Kullanıcının kendini takip etmesi engellenir.
- **MessagesController:** Kullanıcının kendine mesaj göndermesi engellenir; boş mesaj body reddetilir.

7.3 Anti-Forgery Token

Tüm POST formlarında @Html.AntiForgeryToken() ile CSRF token üretilmekte, tüm [HttpPost] action'larında [ValidateAntiForgeryToken] attribute'u ile doğrulanmaktadır. Bu, Cross-Site Request Forgery saldırılarına karşı standart koruma sağlar.

8. Deployment ve CI/CD

8.1 Docker Konteyner Yapısı

Uygulama iki ayrı Docker Compose dosyasıyla hem yerel geliştirme hem de production senaryolarını desteklemektedir:

Dosya	Kullanım Amacı
docker-compose.yml	Yerel geliştirme: Hem PostgreSQL hem app container'ı birlikte başlatır. PostgreSQL'i localhost:5432'ye expose eder. Health check ile app, DB hazır olmadan başlamaz.
docker-compose.prod.yml	Production: Yalnızca app container'ı oluşturur; mevcut shared-db Docker network'ündeki postgres container'ına bağlanır. Port 127.0.0.1:APP_PORT:8080 olarak sadece localhost'a bind edilir; doğrudan public erişim engellenir.

Dockerfile, çok aşamalı (multi-stage) build kullanmaktadır: mcr.microsoft.com/dotnet/sdk:9.0 ile derleme yapılır, mcr.microsoft.com/dotnet/aspnet:9.0 ile küçük runtime image oluşturulur. ASPNETCORE_HTTP_PORTS=8080 ve ENTRYPOINT ["dotnet", "WebParfum.dll"] ile uygulama başlatılır.

8.2 GitHub Actions CI/CD Pipeline

Proje, .github/workflows/deploy.yml dosyasıyla tam otomatik CI/CD pipeline'ına sahiptir. production branch'ine her push yapıldığında veya manuel tetiklendiğinde:

1. appleboy/ssh-action@v1.0.3 ile sunucuya SSH bağlantısı kurulur.
2. /home/parfum/web_parfum/deploy.sh scripti uzaktan çalıştırılır.
3. Deploy script: git pull → docker compose -f docker-compose.prod.yml up -d --build adımlarını gerçekleştirir.

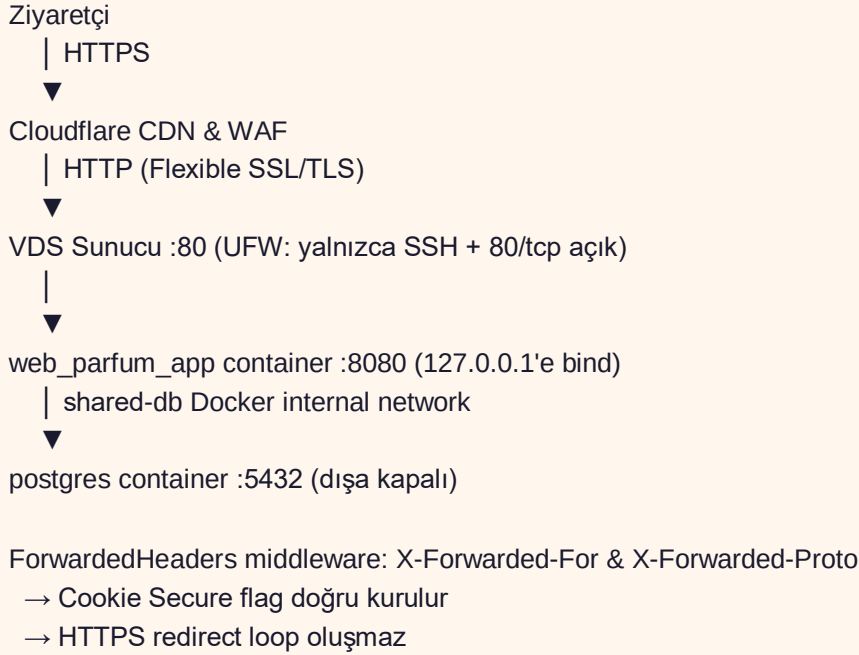
Sunucu IP adresi, SSH kullanıcı adı ve SSH private key, GitHub repository Secrets'larında güvenli biçimde saklanmaktadır (VDS_HOST, VDS_USER, VDS_SSH_KEY).

8.3 Çevre Değişkenleri Yönetimi

.env.example dosyası, geliştiricilerin kendi ortamlarını hızlıca yapılandırabilmesi için şablon görevi görür. Gerçek .env dosyası .gitignore aracılığıyla repository'ye dahil edilmez; production'da doğrudan sunucu üzerinde oluşturulur. Zorunlu değişkenler:

- POSTGRES_PASSWORD — Veritabanı şifresi (production'da güçlü değer zorunlu)
- DB_HOST, DB_PORT, POSTGRES_DB, POSTGRES_USER — Bağlantı parametreleri
- APP_PORT — Uygulamanın dışarıya açık portu
- ASPNETCORE_ENVIRONMENT — Production / Development

8.4 Production Mimarisi



8.5 Canlı Uygulama

Uygulama <https://parfum.avesitesi.xyz> adresinde erişilebilir durumdadır. İlk deployment sırasında ~24.000 parfümün CSV'den okunup veritabanına yazılması yaklaşık 30-60 saniye sürmüştür. Sonraki deploymentlarda (production branch'e push), seed atlanır ve yalnızca uygulama yeniden derlenerek container restart edilir.

9. Teknoloji Yığını Özeti

Teknoloji	Kullanım Amacı
ASP.NET Core 9 MVC	Web framework, MVC pattern, Razor view engine
Entity Framework Core 9.0	ORM, Code-First migrations, LINQ sorguları
Npgsql EF Core Provider 9.0.4	PostgreSQL bağlantısı, pg_trgm / trigram desteği
PostgreSQL 17 Alpine	Veritabanı; GENERATED STORED kolonlar, GIN indeks, unaccent
BCrypt.Net-Next 4.1.0	Şifre hash'leme (work factor=12)
System.Text.Encoding.CodePages	fra_cleaned.csv (ISO-8859-1) okuma
Bootstrap 5.3.3	Responsive grid, bileşenler, modal, nav pills
Bootstrap Icons 1.11.3	İkon seti (600+ simge)
Playfair Display + Inter (Google Fonts)	Tipografi: serif başlıklar + sans-serif gövde
Docker + Docker Compose	Konteyner orkestrasyonu
GitHub Actions	CI/CD pipeline (SSH ile auto-deploy)
Cloudflare	CDN, WAF, HTTPS proxy, DDoS koruması
JavaScript (Vanilla)	AJAX arama, like toggle, autocomplete, star rating widget

10. Sonuç

Koku. projesi, Web Programlama dersinin belirlediği tüm teknik gereksinimleri karşılamakla birlikte birçok açıdan bu gereksinimlerin üzerine çıkmıştır. Proje; EF Core Code-First yaklaşımıyla veri yönetimi, cookie tabanlı kimlik doğrulama ve çok katmanlı yetkilendirme, ViewModel pattern'ıyla veri taşıma, Data Annotations ile form validasyonu gibi temel gereksinimleri eksiksiz yerine getirmiştir.

Bu temel katmanlara ek olarak; tipoya dayanıklı ve aksan-agnostik trigram tabanlı arama motoru, sinyal ağırlıklı hibrit öneri sistemi, AJAX ile anlık UI güncellemeleri, tam işlevsel CI/CD pipeline, Docker konteyner mimarisi ve Cloudflare proxy arkasında gerçek bir VDS sunucusuna deploy gibi üretim kalitesinde bileşenler de geliştirilmiştir.

Sonuç olarak proje; parfüm tutkunları için gerçekten kullanılabilir, ölçeklenebilir ve güvenli bir platform olarak tamamlanmış ve <https://parfum.avesitesi.xyz> adresinde canlıya alınmıştır.